task2-p

January 19, 2025

0.1 Deakin University

0.2 SIG731- Data Wrangling

Name: Surendra Bahadur Rai

Email: s223940212@deakin.com

Student ID: s223940212

0.3 Task 2

This task is related to Module 2 (Sections 2.1-2.4) Learning Practice Exercises. Using the NumPy library and Pandas, I have learned in this unit how to use the NumPy library for data manipulation and efficiently apply data wrangling techniques using library features. This task is Basically Trading BTC Analysis of Historical data from 2023-01-01 up to 2023-12-31 and To Find the Based on Data Investor Status Global Impact of Crypto markets

0.3.1 Objective of the Task

The primary objective is to perform a trading analysis of Bitcoin (BTC) using historical data from January 1, 2023, to December 31, 2023. The analysis aims to:

- Understand BTC price fluctuations over the year.
- Identify significant patterns or anomalies.
- Evaluate the global impact of cryptocurrency markets on investors' behavior and market dynamics.

0.3.2 Q1. Download the daily close BTC-to-USD data, from 2023-01-01 up to 2023-12-31, available at

https://finance.yahoo.com/quote/BTC-USD (the Historical Data tab).

[1]:
```
#Importing Pakage Library

import yfinance as yf
import pandas as pd
import numpy as np
from scipy import stats
import matplotlib.pyplot as plt

import warnings
```

```
warnings.filterwarnings('ignore')
```

[2]:
```
# Declare tiker symbol and start date and end date
ticker_symbol = "BTC-USD"
start_date = "2023-01-01"
end_date = "2023-12-31"
# Fetch the data Using  yfinace libray
btc_data = yf.download(ticker_symbol, start=start_date, end=end_date,␣
 ↪progress=False)
```

Here, set the start date to 2023-01-01 and the end date to 2023-12-31 to download historical data.

[3]:
```
#just print the Head 5 elments
btc_data.head()
```

[3]:
```
Price              Close          High           Low          Open  \
Ticker           BTC-USD       BTC-USD       BTC-USD       BTC-USD
Date
2023-01-01  16625.080078  16630.439453  16521.234375  16547.914062
2023-01-02  16688.470703  16759.343750  16572.228516  16625.509766
2023-01-03  16679.857422  16760.447266  16622.371094  16688.847656
2023-01-04  16863.238281  16964.585938  16667.763672  16680.205078
2023-01-05  16836.736328  16884.021484  16790.283203  16863.472656

Price            Volume
Ticker          BTC-USD
Date
2023-01-01   9244361700
2023-01-02  12097775227
2023-01-03  13903079207
2023-01-04  18421743322
2023-01-05  13692758566
```

The Yfinance library uses a Pandas DataFrame to display data in a tabular view. The headings represent columns, and here, 3 rows of data are being shown.

[4]:
```
#BTC Symbol data shape
btc_data.shape
```

[4]:
```
(364, 5)
```

[5]:
```
# Save to a CSV file (optional)
btc_data.to_csv("BTC_USD_2023.csv")
```

[6]:
```
# Keep only the 'Close' column
btc_close_data = btc_data[['Close']]

# Save to a CSV file only Daily close
```

```
btc_close_data.to_csv("BTC_USD_Daily_Close_2023.csv")
```

[7]:
```
btc_close_data
```

[7]:
```
Price              Close
Ticker           BTC-USD
Date
2023-01-01  16625.080078
2023-01-02  16688.470703
2023-01-03  16679.857422
2023-01-04  16863.238281
2023-01-05  16836.736328
…                    …
2023-12-26  42520.402344
2023-12-27  43442.855469
2023-12-28  42627.855469
2023-12-29  42099.402344
2023-12-30  42156.902344

[364 rows x 1 columns]
```

Here, data only shown Daily Trading close Colum and Date & Heading 3 rows showing

0.3.3 Q2. Use numpy.genfromtxt or numpy.loadtxt to read the above BTC-to-USD data as a numpy vector named rates.

[8]:
```
# Use numpy.genfromtxt Reading Historical data
rates = np.genfromtxt("BTC_USD_2023.csv", delimiter=",", skip_header=3,␣
 ↪usecols=1)

# Print the first few rates to verify
print("Rates as a NumPy vector:")
print(rates)
```

```
Rates as a NumPy vector:
[16625.08007812 16688.47070312 16679.85742188 16863.23828125
 16836.73632812 16951.96875    16955.078125   17091.14453125
 17196.5546875  17446.29296875 17934.89648438 18869.58789062
 19909.57421875 20976.29882812 20880.79882812 21169.6328125
 21161.51953125 20688.78125    21086.79296875 22676.55273438
 22777.625      22720.41601562 22934.43164062 22636.46875
 23117.859375   23032.77734375 23078.72851562 23031.08984375
 23774.56640625 22840.13867188 23139.28320312 23723.76953125
 23471.87109375 23449.32226562 23331.84765625 22955.66601562
 22760.109375   23264.29101562 22939.3984375  21819.0390625
 21651.18359375 21870.875      21788.203125   21808.1015625
 22220.8046875  24307.84179688 23623.47460938 24565.6015625
```

24641.27734375 24327.64257812 24829.1484375 24436.35351562
24188.84375   23947.4921875 23198.12695312 23175.375
23561.21289062 23522.87109375 23147.35351562 23646.55078125
23475.46679688 22362.6796875 22353.34960938 22435.51367188
22429.7578125 22219.76953125 21718.08007812 20363.02148438
20187.24414062 20632.41015625 22163.94921875 24197.53320312
24746.07421875 24375.9609375 25052.7890625 27423.9296875
26965.87890625 28038.67578125 27767.23632812 28175.81640625
27307.4375   28333.97265625 27493.28515625 27494.70703125
27994.33007812 27139.88867188 27268.13085938 28348.44140625
28033.5625   28478.484375   28411.03515625 28199.30859375
27790.22070312 28168.08984375 28177.984375   28044.140625
27925.859375   27947.79492188 28333.05078125 29652.98046875
30235.05859375 30139.05273438 30399.06640625 30485.69921875
30318.49609375 30315.35546875 29445.04492188 30397.55273438
28822.6796875 28245.98828125 27276.91015625 27817.5
27591.38476562 27525.33984375 28307.59765625 28422.70117188
29473.78710938 29340.26171875 29248.48828125 29268.80664062
28091.56835938 28680.53710938 29006.30859375 28847.7109375
29534.38476562 28904.62304688 28454.97851562 27694.2734375
27658.77539062 27621.75585938 27000.7890625 26804.99023438
26784.078125   26930.63867188 27192.69335938 27036.65039062
27398.80273438 26832.20898438 26890.12890625 27129.5859375
26753.82617188 26851.27734375 27225.7265625 26334.81835938
26476.20703125 26719.29101562 26868.35351562 28085.64648438
27745.88476562 27702.34960938 27219.65820312 26819.97265625
27249.58984375 27075.12890625 27119.06640625 25760.09765625
27238.78320312 26345.99804688 26508.21679688 26480.375
25851.24023438 25940.16796875 25902.5   25918.72851562
25124.67578125 25576.39453125 26327.46289062 26510.67578125
26336.21289062 26851.02929688 28327.48828125 30027.296875
29912.28125   30695.46875   30548.6953125 30480.26171875
30271.13085938 30688.1640625 30086.24609375 30445.3515625
30477.25195312 30590.078125   30620.76953125 31156.43945312
30777.58203125 30514.16601562 29909.33789062 30342.265625
30292.54101562 30171.234375   30414.47070312 30620.95117188
30391.64648438 31476.04882812 30334.06835938 30295.80664062
30249.1328125 30145.88867188 29856.5625   29913.92382812
29792.015625   29908.74414062 29771.80273438 30084.5390625
29176.91601562 29227.390625   29354.97265625 29210.68945312
29319.24609375 29356.91796875 29275.30859375 29230.11132812
29675.73242188 29151.95898438 29178.6796875 29074.09179688
29042.12695312 29041.85546875 29180.578125   29765.4921875
29561.49414062 29429.59179688 29397.71484375 29415.96484375
29282.9140625 29408.44335938 29170.34765625 28701.77929688
26664.55078125 26049.55664062 26096.20507812 26189.58398438
26124.140625   26031.65625   26431.640625   26162.37304688
26047.66796875 26008.46289062 26089.69335938 26106.15039062

```
 27727.39257812 27297.265625   25931.47265625 25800.72460938
 25868.79882812 25969.56640625 25812.41601562 25779.98242188
 25753.23632812 26240.1953125  25905.65429688 25895.67773438
 25832.2265625  25162.65429688 25833.34375    26228.32421875
 26539.67382812 26608.69335938 26568.28125    26534.1875
 26754.28125    27211.1171875  27132.0078125  26567.6328125
 26579.56835938 26579.390625   26256.82617188 26298.48046875
 26217.25       26352.71679688 27021.546875   26911.72070312
 26967.91601562 27983.75       27530.78515625 27429.97851562
 27799.39453125 27415.91210938 27946.59765625 27968.83984375
 27935.08984375 27583.67773438 27391.01953125 26873.3203125
 26756.79882812 26862.375      26861.70703125 27159.65234375
 28519.46679688 28415.74804688 28328.34179688 28719.80664062
 29682.94921875 29918.41210938 29993.89648438 33086.234375
 33901.52734375 34502.8203125  34156.6484375  33909.80078125
 34089.57421875 34538.48046875 34502.36328125 34667.78125
 35437.25390625 34938.2421875  34732.32421875 35082.1953125
 35049.35546875 35037.37109375 35443.5625     35655.27734375
 36693.125      37313.96875    37138.05078125 37054.51953125
 36502.35546875 35537.640625   37880.58203125 36154.76953125
 36596.68359375 36585.703125   37386.546875   37476.95703125
 35813.8125     37432.33984375 37289.62109375 37720.28125
 37796.79296875 37479.12109375 37254.16796875 37831.0859375
 37858.4921875  37712.74609375 38688.75       39476.33203125
 39978.390625   41980.09765625 44080.6484375  43746.4453125
 43292.6640625  44166.6015625  43725.984375   43779.69921875
 41243.83203125 41450.22265625 42890.7421875  43023.97265625
 41929.7578125  42240.1171875  41364.6640625  42623.5390625
 42270.52734375 43652.25       43869.15234375 43997.90234375
 43739.54296875 43016.1171875  43613.140625   42520.40234375
 43442.85546875 42627.85546875 42099.40234375 42156.90234375]
```

This all Historical data of BTC Symbol to 2023-01-01 and the end date to 2023-12-31 in numpy vector, and avoiding header skip_header=3

[9]:
```
# Use numpy.genfromtxt Reading Daily close Symbol oly skipping Header 3 rows␣
 ↪only for data
rates1 = np.genfromtxt("BTC_USD_Daily_Close_2023.csv", delimiter=",",␣
 ↪skip_header=3, usecols=1)

# Printting the Numpy Array vector
print("Rates as a NumPy vector:")
print(rates1)
```

```
Rates as a NumPy vector:
[16625.08007812 16688.47070312 16679.85742188 16863.23828125
 16836.73632812 16951.96875    16955.078125   17091.14453125
 17196.5546875  17446.29296875 17934.89648438 18869.58789062
```

19909.57421875 20976.29882812 20880.79882812 21169.6328125
21161.51953125 20688.78125 21086.79296875 22676.55273438
22777.625 22720.41601562 22934.43164062 22636.46875
23117.859375 23032.77734375 23078.72851562 23031.08984375
23774.56640625 22840.13867188 23139.28320312 23723.76953125
23471.87109375 23449.32226562 23331.84765625 22955.66601562
22760.109375 23264.29101562 22939.3984375 21819.0390625
21651.18359375 21870.875 21788.203125 21808.1015625
22220.8046875 24307.84179688 23623.47460938 24565.6015625
24641.27734375 24327.64257812 24829.1484375 24436.35351562
24188.84375 23947.4921875 23198.12695312 23175.375
23561.21289062 23522.87109375 23147.35351562 23646.55078125
23475.46679688 22362.6796875 22353.34960938 22435.51367188
22429.7578125 22219.76953125 21718.08007812 20363.02148438
20187.24414062 20632.41015625 22163.94921875 24197.53320312
24746.07421875 24375.9609375 25052.7890625 27423.9296875
26965.87890625 28038.67578125 27767.23632812 28175.81640625
27307.4375 28333.97265625 27493.28515625 27494.70703125
27994.33007812 27139.88867188 27268.13085938 28348.44140625
28033.5625 28478.484375 28411.03515625 28199.30859375
27790.22070312 28168.08984375 28177.984375 28044.140625
27925.859375 27947.79492188 28333.05078125 29652.98046875
30235.05859375 30139.05273438 30399.06640625 30485.69921875
30318.49609375 30315.35546875 29445.04492188 30397.55273438
28822.6796875 28245.98828125 27276.91015625 27817.5
27591.38476562 27525.33984375 28307.59765625 28422.70117188
29473.78710938 29340.26171875 29248.48828125 29268.80664062
28091.56835938 28680.53710938 29006.30859375 28847.7109375
29534.38476562 28904.62304688 28454.97851562 27694.2734375
27658.77539062 27621.75585938 27000.7890625 26804.99023438
26784.078125 26930.63867188 27192.69335938 27036.65039062
27398.80273438 26832.20898438 26890.12890625 27129.5859375
26753.82617188 26851.27734375 27225.7265625 26334.81835938
26476.20703125 26719.29101562 26868.35351562 28085.64648438
27745.88476562 27702.34960938 27219.65820312 26819.97265625
27249.58984375 27075.12890625 27119.06640625 25760.09765625
27238.78320312 26345.99804688 26508.21679688 26480.375
25851.24023438 25940.16796875 25902.5 25918.72851562
25124.67578125 25576.39453125 26327.46289062 26510.67578125
26336.21289062 26851.02929688 28327.48828125 30027.296875
29912.28125 30695.46875 30548.6953125 30480.26171875
30271.13085938 30688.1640625 30086.24609375 30445.3515625
30477.25195312 30590.078125 30620.76953125 31156.43945312
30777.58203125 30514.16601562 29909.33789062 30342.265625
30292.54101562 30171.234375 30414.47070312 30620.95117188
30391.64648438 31476.04882812 30334.06835938 30295.80664062
30249.1328125 30145.88867188 29856.5625 29913.92382812
29792.015625 29908.74414062 29771.80273438 30084.5390625

```
29176.91601562 29227.390625   29354.97265625 29210.68945312
29319.24609375 29356.91796875 29275.30859375 29230.11132812
29675.73242188 29151.95898438 29178.6796875  29074.09179688
29042.12695312 29041.85546875 29180.578125   29765.4921875
29561.49414062 29429.59179688 29397.71484375 29415.96484375
29282.9140625  29408.44335938 29170.34765625 28701.77929688
26664.55078125 26049.55664062 26096.20507812 26189.58398438
26124.140625   26031.65625    26431.640625   26162.37304688
26047.66796875 26008.46289062 26089.69335938 26106.15039062
27727.39257812 27297.265625   25931.47265625 25800.72460938
25868.79882812 25969.56640625 25812.41601562 25779.98242188
25753.23632812 26240.1953125  25905.65429688 25895.67773438
25832.2265625  25162.65429688 25833.34375    26228.32421875
26539.67382812 26608.69335938 26568.28125    26534.1875
26754.28125    27211.1171875  27132.0078125  26567.6328125
26579.56835938 26579.390625   26256.82617188 26298.48046875
26217.25       26352.71679688 27021.546875   26911.72070312
26967.91601562 27983.75       27530.78515625 27429.97851562
27799.39453125 27415.91210938 27946.59765625 27968.83984375
27935.08984375 27583.67773438 27391.01953125 26873.3203125
26756.79882812 26862.375      26861.70703125 27159.65234375
28519.46679688 28415.74804688 28328.34179688 28719.80664062
29682.94921875 29918.41210938 29993.89648438 33086.234375
33901.52734375 34502.8203125  34156.6484375  33909.80078125
34089.57421875 34538.48046875 34502.36328125 34667.78125
35437.25390625 34938.2421875  34732.32421875 35082.1953125
35049.35546875 35037.37109375 35443.5625     35655.27734375
36693.125      37313.96875    37138.05078125 37054.51953125
36502.35546875 35537.640625   37880.58203125 36154.76953125
36596.68359375 36585.703125   37386.546875   37476.95703125
35813.8125     37432.33984375 37289.62109375 37720.28125
37796.79296875 37479.12109375 37254.16796875 37831.0859375
37858.4921875  37712.74609375 38688.75       39476.33203125
39978.390625   41980.09765625 44080.6484375  43746.4453125
43292.6640625  44166.6015625  43725.984375   43779.69921875
41243.83203125 41450.22265625 42890.7421875  43023.97265625
41929.7578125  42240.1171875  41364.6640625  42623.5390625
42270.52734375 43652.25       43869.15234375 43997.90234375
43739.54296875 43016.1171875  43613.140625   42520.40234375
43442.85546875 42627.85546875 42099.40234375 42156.90234375]
```

Only Closed BTC Symbol to 2023-01-01 and the end date to 2023-12-31 in numpy vector

[ ]:

0.3.4 Q3. For the fourth quarter of the year only (Q4 2023; days 274–365 inclusive),determine and display(in a readable manner) the following aggregates:

- arithmetic mean
- minimum
- the first quartile,
- median,
- the third quartile,
- maximum,
- standard deviation,
- interquartile range.

[10]:
```
# Filter the data for the fourth quarter (q4 2023) (days 274-365 inclusive)
q4_data = btc_data.loc['2023-10-01':'2023-12-31']

# Extract the 'Close' prices
q4_close = q4_data['Close']

# Calculate the desired aggregates using scipy and numpy
aggregates = {
    'Arithmetic Mean': np.mean(q4_close),
    'Minimum': np.min(q4_close),
    'First Quartile (Q1)': np.percentile(q4_close, 25),
    'Median': np.median(q4_close),
    'Third Quartile (Q3)': np.percentile(q4_close, 75),
    'Maximum': np.max(q4_close),
    'Standard Deviation': np.std(q4_close),
    'Interquartile Range (IQR)': stats.iqr(q4_close)  # IQR using scipy
}

# Display the aggregates:
print("Q4 2023 BTC-USD Aggregates: \n")
for key, value in aggregates.items():
    print(f"##   {key}: {float(value):.2f}")
```

```
Q4 2023 BTC-USD Aggregates:

##   Arithmetic Mean: 36230.84
##   Minimum: 26756.80
##   First Quartile (Q1): 33493.88
##   Median: 36693.12
##   Third Quartile (Q3): 41954.93
##   Maximum: 44166.60
##   Standard Deviation: 5603.84
##   Interquartile Range (IQR): 8461.05
```

- Filter the data for the fourth quarter (Q4 2023) (days 274–365 inclusive)
- Extract the ‘Close’ prices
- Calculate the desired aggregates using scipy and numpy

- Q4 2023 BTC-USD Aggregates values

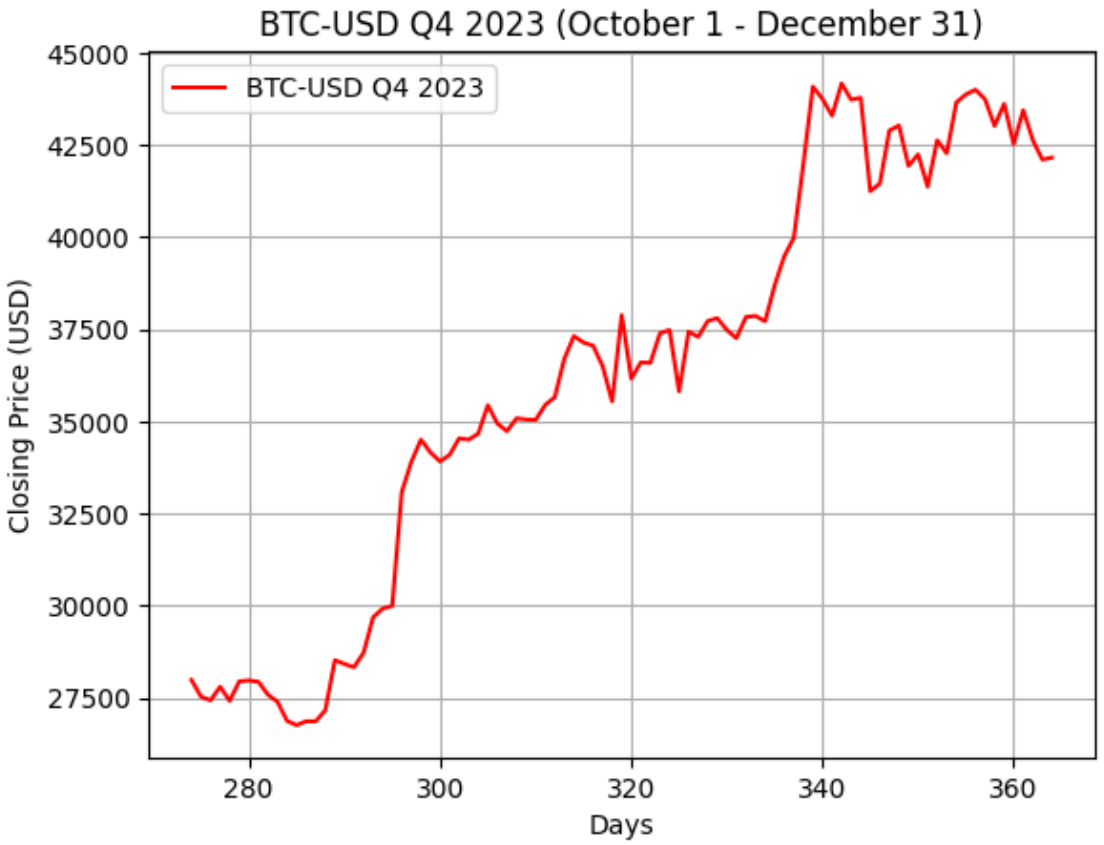
0.3.5 Q4. Call matplotlib.pyplot.plot(days, rates, <...further_arguments...>) to draw the Q4 2023 data (with 274 denoting 1 October), using red solid line segments.

[11]:
```
# Filter the data for the fourth quarter (Q4 2023) (days 274-365 inclusive)
q4_data = btc_data.loc['2023-10-01':'2023-12-31']

# Extract the 'Close' prices for Q4
q4_close = q4_data['Close']

# days only in array where day 274 corresponds to October 1, 2023
days = np.arange(274, 274 + len(q4_close))

# Plot the data
plt.plot(days, q4_close, 'r-', label='BTC-USD Q4 2023')
plt.title('BTC-USD Q4 2023 (October 1 - December 31)')
plt.xlabel('Days')
plt.ylabel('Closing Price (USD)')
plt.grid(True)
plt.legend()
plt.show()
```



- Filter the data for the fourth quarter (Q4 2023) (days 274–365 inclusive)
- Extract the 'Close' prices for Q4
- Create the 'days' in arry list where day 274 corresponds to October 1, 2023
- using matplot lib libray & Plot line graph of data only BTC-USD Q4 2023 (October 1 - December 31)

[ ]:

0.3.6 Q5. Determine the day numbers (with 274 denoting 1 October) with the lowest and highest observed prices in Q4 2023. Below is an example of the lowest and highest price days in Q3 2023.

[12]:
```
# Filter the data for the fourth quarter (Q4 2023) (days 274-365 inclusive)
q4_data = btc_data.loc['2023-10-01':'2023-12-31']

# Extract the 'Close' prices for Q4
q4_close = q4_data['Close']

# Find the 'days' array where day 274 corresponds to October 1, 2023
days = np.arange(274, 274 + len(q4_close))

# Find the day with the lowest price
min_price = np.min(q4_close)
min_day = days[np.argmin(q4_close)]

# Find the day with the highest price
max_price = np.max(q4_close)
max_day = days[np.argmax(q4_close)]

# Print the results
print(f"## Lowest price was on day {min_day} ({min_price:.2f}).")
print(f"## Highest price was on day {max_day} ({max_price:.2f}).")
```

```
## Lowest price was on day 285 (26756.80).
## Highest price was on day 342 (44166.60).
```

- Filter the data for the fourth quarter (Q4 2023) (days 274–365 inclusive)
- Extract the 'Close' prices for Q4
- Find the 'days' array where day 274 corresponds to October 1, 2023
- Find the day with the lowest price using numpy.min of the close price
- Find the day with the highest price using numpy.max of the close price
- Print the results

0.3.7 Q6. Using matplotlib.pyplot.boxplot, draw a horizontal box-and-whisker plot for the Q4 2023 daily price increases/decreases as obtained by a call to numpy.diff.

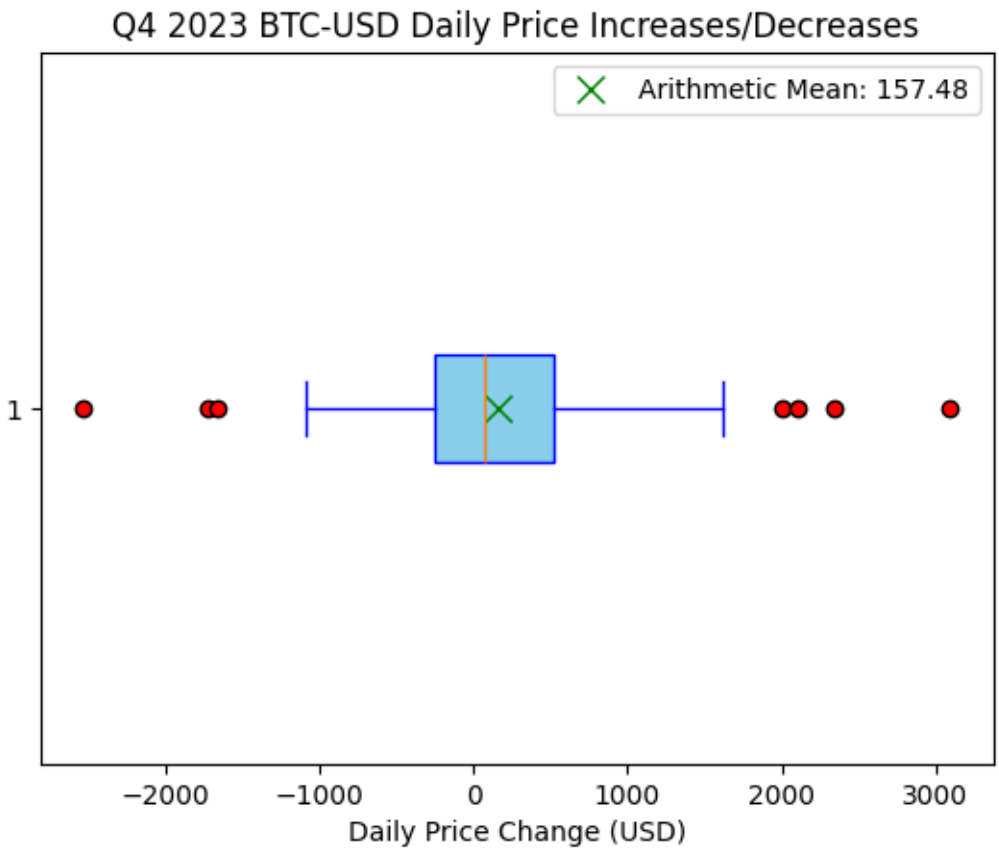
[13]:
```
# Filter the data for the fourth quarter (Q4 2023)
q4_data = btc_data.loc['2023-10-01':'2023-12-31']

# Find the close column
q4_close = q4_data['Close'].values

# Calculate the daily price increases/decreases using numpy.diff()
daily_changes = np.diff(q4_close,axis=0)

# Create the horizontal boxplot for the daily price changes
plt.boxplot(daily_changes, vert=False, patch_artist=True,␣
 ↪boxprops=dict(facecolor='skyblue', color='blue'),
            whiskerprops=dict(color='blue'), capprops=dict(color='blue'),␣
 ↪flierprops=dict(markerfacecolor='red', marker='o'))

# Mark the arithmetic mean on the boxplot with a green "x"
mean_change = np.mean(daily_changes)
plt.plot(mean_change, 1, 'gx', markersize=10, label=f'Arithmetic Mean:␣
 ↪{mean_change:.2f}')
# Add labels and title
plt.xlabel('Daily Price Change (USD)')
plt.title('Q4 2023 BTC-USD Daily Price Increases/Decreases')
plt.legend()
plt.show()
```



- Filter the data for the fourth quarter (Q4 2023)
- Find the close column price
- Calculate the daily price increases/decreases using numpy.diff()
- Create the horizontal boxplot for the daily price changes
- Mark the arithmetic mean on the boxplot with a green "x"
- Add labels and title

Learning outcome of this Question is Box plot display:

Median: The central line in the box.

Interquartile Range (IQR): The box's length.

Whiskers: Extend to the smallest and largest data points within 1.5×IQR.

Outliers: showing data point of the whiskers.

[ ]:

0.3.8 Q7. Count (programmatically,using the vectorised relational operators from numpy) how many outliers the boxplot contains (for the definition of an outlier, consult Section 2.3 of our learning materials on the unit site or Section 5.1 in the Book)

[14]:
```
# Filter the data
q4_data = btc_data.loc['2023-10-01':'2023-12-31']

# Find the 'Close' prices for Q4
q4_close = q4_data['Close'].values

# Calculate the daily price increases/decreases using numpy.diff()
daily_changes = np.diff(q4_close,axis=0)

#Compute Q1, Q3, and IQR
Q1 = np.percentile(daily_changes, 25)
Q3 = np.percentile(daily_changes, 75)
IQR = Q3 - Q1

# Calculate the lower and upper bounds for outliers
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Count the outliers using numpy's relational operators
outliers = (daily_changes < lower_bound) | (daily_changes > upper_bound)
num_outliers = np.sum(outliers)

# Output the result
print(f"Number of outliers: {num_outliers}")
```

```
Number of outliers: 7
```

Heres did this step

- Filter the data
- Find the 'Close' prices for Q4
- Compute Q1, Q3, and IQR
- Count the outliers using numpy's relational operators
- Output the result

Outliers in a boxplot are defined as data points: $Q1 - 1.5 \times IQR$ (lower bound)

or

$Q3 + 1.5 \times IQR$ (upper bound), where

- $Q1$: First quartile (25th percentile).
- $Q3$: Third quartile (75th percentile).
- IQR: Interquartile range ($Q3 - Q1$).

***Discription:**

Outliers in this context represent unusually large price increases or decreases in Bitcoin's daily closing prices during Q4 2023. These could be caused by:

- **Market Volatility**: Significant changes in demand or supply, often driven by external events like regulatory announcements, technological advancements, or macroeconomic factors.
- **Investor Behavior**: Sudden large-scale buying or selling by institutional or retail investors.
- **Global Events**: News impacting the cryptocurrency market, such as government bans, adoption by major companies, or significant hacks.
- **Low Liquidity**: During certain periods, lower market liquidity can amplify price movements, leading to outliers.

[ ]: